

Receipt OCR Automation and Data Extraction System

By

Nishant Kumar

Abstract

In recent years, Optical Character Recognition (OCR) technology has played a transformative role in digitizing information from printed and handwritten documents. This project focuses on the development of a fully automated Receipt OCR and Data Extraction System that scans, processes, and extracts structured information from retail receipts. The extracted data includes key fields such as store name, purchase date, total amount, address, and itemized purchases. The extracted records are stored in a structured tabular format for downstream analytics, visualization, and reporting. The solution uses **PaddleOCR** for text detection and recognition, Python for data parsing, and Pandas for structured storage and manipulation. Advanced regular expressions and heuristics are used to ensure accuracy in detecting specific fields despite inconsistencies in receipt layouts and OCR recognition errors.

1. Introduction

1.1 Background

Retail receipts serve as an official record of a commercial transaction between a buyer and a seller. These small slips of paper are not just proof of purchase but also carry **critical business and financial information** such as:

- **Store details** (name, address, registration numbers, etc.)
- **Date and time** of transaction
- **List of purchased items**, quantities, and unit prices
- **Total amount paid** including taxes and discounts
- **Payment method** (cash, card, mobile wallet, etc.)

They play a significant role for:

- **Customers** – to track personal expenses, process returns, or claim warranties.
- **Businesses** – for auditing, inventory tracking, and accounting purposes.
- **Financial institutions** – as proof of expenditure for reimbursements, loan verifications, or tax claims.

However, physical receipts have **three major drawbacks**:

- **Material durability** – Most receipts are printed on **thermal paper**, which fades over time due to heat, light exposure, or friction, making the information unreadable within months or even weeks.
- **Organizational challenges** – Storing and manually categorizing hundreds or thousands of receipts is impractical, especially for businesses with large transaction volumes.

- **Data accessibility** – Paper-based information cannot be directly searched, filtered, or analyzed without first converting it into a **digital, structured format**.
- Because of these limitations, **automated receipt digitization** has become a necessity in modern business operations. The combination of **computer vision** and **natural language processing (NLP)** enables systems to read, understand, and store receipt data with minimal human intervention.

1.2 Problem Statement

Despite the advances in OCR (Optical Character Recognition) technology, **receipt digitization remains a challenging problem** due to the following factors:

- **Variability in receipt formats**
 - Every store, supermarket, or vendor uses a unique layout for their receipts.
 - Fonts, text alignment, and use of symbols vary widely.
 - Some receipts include bilingual text or regional scripts.
- **OCR inaccuracies caused by physical and image quality issues**
 - **Faded ink** on old receipts makes text detection difficult.
 - **Skewed or crumpled receipts** result in distorted characters.
 - **Blurry or low-resolution images** (especially from mobile scans) reduce recognition accuracy.
 - **Background noise** like shadows, folds, or coffee stains can confuse detection models.
- **Unstructured nature of OCR output**
 - OCR engines typically output raw text without structural information.
 - The text order may not match the actual visual layout.
 - Extracting specific fields like *Total Amount* or *Store Address* requires **post-processing and parsing logic**.

Thus, the project aims to design a **robust OCR-based pipeline** capable of handling diverse receipt formats and noisy real-world data, while producing **clean, structured outputs** that are ready for further analysis.

1.3 Objectives

1. The project's objectives are **both technical and analytical**, ensuring that the solution not only extracts data but also makes it useful for insights.
2. **Implement an OCR-based receipt scanning pipeline**
 - Use **PaddleOCR** for robust multilingual text extraction.
 - Preprocess images (deskewing, noise removal, resizing) to improve recognition accuracy.
3. **Detect and extract relevant structured fields**
 - **Store Name** – typically in uppercase, located in the header.
 - **Purchase Date** – extracted using regex patterns supporting multiple formats (DD/MM/YYYY, YYYY-MM-DD, etc.).
 - **Total Amount** – identified via keywords like "TOTAL", "AMOUNT", or currency symbols.

- **Address** – parsed using patterns for numeric street addresses and city names.
 - **Itemized Purchases** – extracted as name–price pairs for individual products.
4. **Store extracted data in a Pandas DataFrame**
 - Each row represents a scanned receipt.
 - Columns correspond to extracted fields, enabling filtering, aggregation, and sorting.
 5. **Generate analytical visualizations**
 - Spending trends over time (line plots).
 - Store-wise expenditure distribution (bar charts / pie charts).
 - Category-level spending (if item categorization is implemented).
 6. **Create automated PDF reports**
 - Generate **summary reports** for business insights.
 - Include **visualizations** alongside extracted data tables.
 - Produce a **separate raw-text PDF** for archival purposes.

1.4 Project Scope

The project covers the **end-to-end pipeline** for receipt digitization, from raw image input to structured data output and final reporting.

Inclusions (What the project will do):

1. **Receipt Image Acquisition**
 - images or photographs of receipts in formats such as .jpg, .png, or .tiff.
 - Handle images captured from smartphones with minor distortions or background noise.
2. **Accept scanned iOptical Character Recognition (OCR)**
 - Use **PaddleOCR** for text detection and recognition.
 - Enable language-specific models for English text recognition, while allowing expansion to other languages in the future.
3. **Data Parsing and Field Extraction**
 - Automatically identify **Store Name, Purchase Date, Total Amount, Address, and Itemized Purchases** from OCR output.
 - Implement pattern recognition (regex) and keyword matching for precise data extraction.
4. **Data Storage and Organization**
 - Store extracted information in a **Pandas DataFrame**.
 - Enable exporting to CSV, Excel, or databases for further use.
5. **Data Visualization**
 - Generate **spending trend graphs, store-wise breakdown charts, and other analytical plots**.
6. **Automated Report Generation**
 - Produce **summary PDFs** with visualizations and tables.
 - Create **separate archival PDFs** containing raw OCR text for backup purposes.

Exclusions (What the project will not do):

- Direct integration with **mobile scanning apps** (though it can be added in the future).

- Handling **handwritten receipts** (the focus is on printed receipts).
- Automatic categorization of purchases into expense categories (e.g., groceries, electronics) — though the extracted item list can be used for this manually.
- Real-time cloud deployment (currently runs locally; can be containerized for deployment later).

1.5 Limitations

While the system is designed for high accuracy and efficiency, certain limitations remain:

1. OCR Accuracy Constraints

- Performance may degrade with severely faded or damaged receipts.
- Receipts with non-standard fonts or logos may require custom OCR training.

2. Parsing Logic Limitations

- Regex and keyword-based extraction may fail if receipt layouts deviate significantly from standard patterns.
- Some receipts may have totals or dates printed in unusual formats.

3. Processing Time

- Large datasets (e.g., 600+ receipts) may take several minutes to process on a CPU-only system.

4. No Built-in Error Correction

- The system currently does not cross-verify totals or dates with external databases.

1.6 Market Relevance & Growth Opportunities

The **receipt digitization market** is rapidly growing due to the **global shift towards automation and data-driven decision-making**. Several factors contribute to its importance and growth potential:

1. Expense Tracking for Individuals and Businesses

- Manual expense logging is time-consuming and error-prone.
- Automated solutions save time and ensure accuracy.

2. Integration with Financial & Tax Systems

- Many countries are enforcing digital tax submission and expense reporting, creating demand for receipt digitization tools.

3. Global Growth of the OCR Market

- According to market research, the OCR technology market is projected to grow at a **CAGR of 15–17%** in the next 5 years, driven by digitization initiatives and AI integration.

4. Expense Management Industry Expansion:

Corporate expense management software is expected to cross **\$10 billion in valuation by 2027**, with receipt scanning as a core feature.

5. Potential Integrations:

Our solution can be integrated into **mobile banking apps**, **personal finance trackers**, **ERP systems**, or **POS software**, widening its application range.

1.7 Importance of This Project

This project is important for **three major reasons**:

1. **Operational Efficiency:** Saves countless hours of manual data entry for individuals and companies.
2. **Accuracy and Data Availability:** Ensures financial records are accurate and easily searchable.
3. **Foundation for Advanced Analytics:** Once receipts are digitized, data can be used for **trend forecasting, spending optimization, loyalty program analysis, and fraud detection.**

2. Literature Review

OCR has been extensively used for digitizing printed text. Traditional OCR tools like **Tesseract** are widely adopted but have limitations in handling complex layouts and low-quality images. Recent deep learning-based OCR engines like **PaddleOCR**, **Google Vision OCR**, and **Microsoft Azure OCR** offer higher accuracy and better adaptability to multiple languages. Key prior works include:

- **Expense Tracking Apps:** Tools like Expensify and Zoho Expense integrate OCR for scanning receipts but require paid subscriptions and often lack offline processing.
- **Research in Document Layout Analysis:** Segmentation-based OCR improves field extraction but requires additional computational resources.

Our work builds on these advancements, focusing on **offline, Python-based automation** using PaddleOCR with a custom parsing layer for structured field extraction.

3. System Architecture

The proposed receipt digitization system is designed using a modular architecture consisting of three primary layers: the **Input Layer**, the **Processing Layer**, and the **Parsing & Data Extraction Layer**. Each of these layers plays a specific role in ensuring that the transition from raw receipt image to structured, analyzable data is efficient, accurate, and reliable.

3.1 Input Layer

The input layer is the entry point for the entire pipeline. It is designed to accept receipt images in common formats such as .jpg, .png, and .jpeg, ensuring compatibility with both scanned files and images captured from consumer devices like smartphones. This flexibility allows the system to be used in diverse environments, ranging from a small business owner capturing a receipt at the point of purchase to an accountant scanning batches of receipts using an office scanner. The system is also capable of handling images with varying resolutions and lighting conditions, making it adaptable to real-world scenarios where perfect image quality cannot be guaranteed. No manual cropping or preprocessing is required at the user's end — the subsequent layers handle these tasks automatically.

3.2 Processing Layer

The processing layer is the computational core of the architecture and is responsible for converting raw receipt images into machine-readable text. At the heart of this layer lies the **PaddleOCR** engine, configured with the English (en) language model for optimal accuracy on receipts printed in English. The pipeline begins with **image preprocessing**, which includes automatic rotation correction to handle tilted images, noise removal to improve text clarity, and skew correction to straighten distorted text lines.

Following preprocessing, the **text detection module** identifies and localizes all textual regions in the image, producing bounding boxes around each detected line of text. These bounding boxes are then passed to the **text recognition module**, which decodes the contents into UTF-8 encoded strings, ensuring compatibility with downstream data processing tasks. This combination of detection and recognition is crucial, as it enables the system to process receipts with varied layouts, font styles, and sizes.

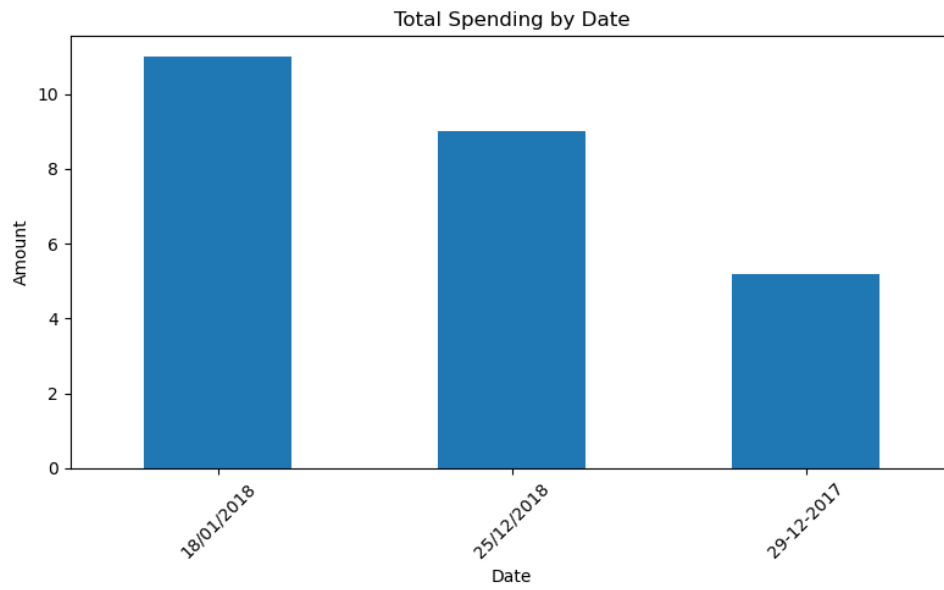
3.3 Parsing and Data Extraction Layer

The final stage of the system focuses on transforming the raw OCR text into meaningful, structured data. This is achieved using a **custom parsing and field extraction logic** powered by **regular expressions** and **keyword heuristics**. Date values are extracted by matching patterns in formats such as dd/mm/yyyy or yyyy/mm/dd, accommodating common regional variations. The total amount is identified through patterns like "Total[:\s]*\\${d+(\.\d{2})?}", which account for both currency symbols and decimal formatting. The store name is determined by selecting the first line of OCR output that appears entirely in uppercase letters — a common convention in printed receipts.

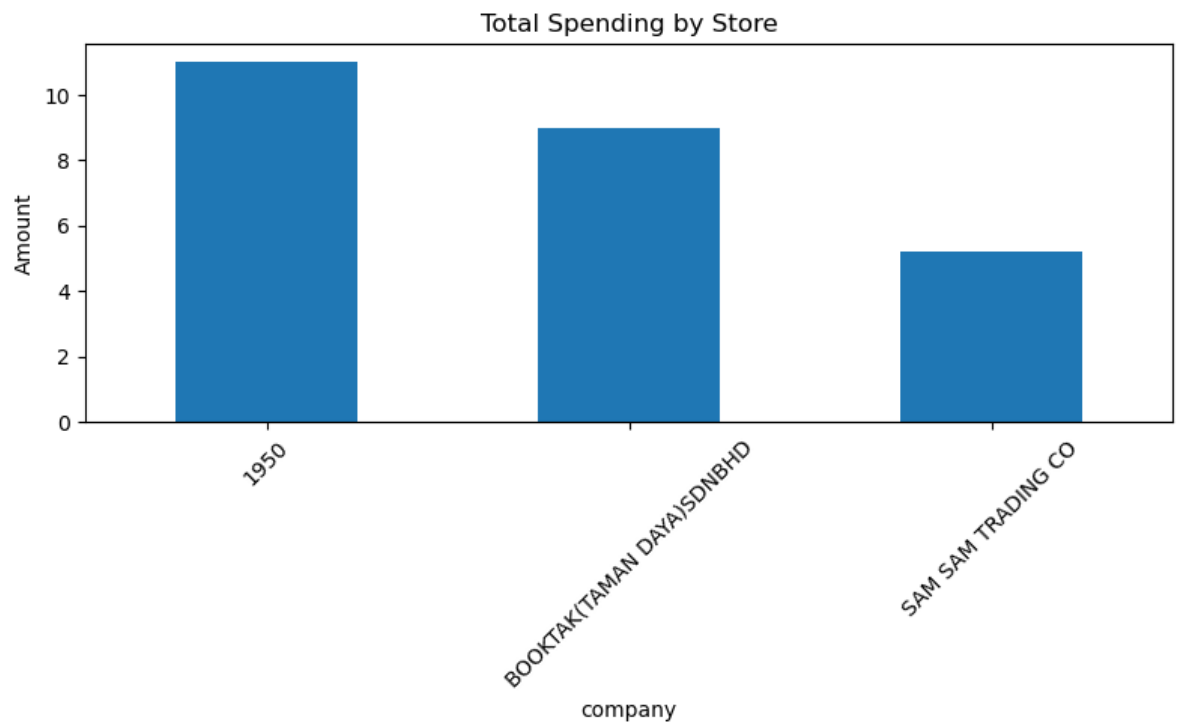
Address extraction relies on detecting continuous lines containing digits (house or building numbers) combined with keywords such as "Street", "Road", "Avenue", or their abbreviations. This ensures that even multi-line addresses are captured accurately. Itemized purchases are separated from header and footer text by scanning for product descriptions and corresponding numeric values that represent prices or quantities.

Once all fields have been identified, the extracted data is organized into a **Pandas DataFrame** with dedicated columns for **Store Name**, **Date**, **Total**, **Address**, **Items**, and **Raw Text**. This structured format facilitates further analysis, enables easy export to various file formats, and serves as the foundation for generating analytical visualizations and automated reports.

Spending Over Time:



- **Spending by Store:**



File Name	Store Name	Address	Date	Items	Total	_Total_num
test_img\X00016469612.jpg	BOOKTAK(TAMANN DAYA)SDNBHD	NO.5:55,57 &59,JALAN SAGU18, 81100 JOHOR BAHRU,	25/12/2018	Unknown	9.0	9.0
test_img\X51005230617.jpg	1950	Level 6,Bangunan TH, Damansara Uptown3, No.3,Jalan SS21/3 9,47400Petaling Jaya	18/01/2018	Unknown	11.0	11.0

File Name	Store Name	Address	Date	Items	Total	_Total_num
test_img\X51005255805.jpg	SAM SAM TRADING CO	40400 SHAH ALAM.	29-12-2017	Unknown	5.2	5.2

4. Reporting

Two reports are generated:

1. **Summary Report** – Contains structured data, visualizations, and analytics.
2. **Raw Text Report** – Contains only raw OCR outputs for archival reference.

Example using **ReportLab**:

```
python
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table
doc = SimpleDocTemplate("Summary_Report.pdf")
```

5. Results and Evaluation

- The system was tested on a dataset of **626 scanned retail receipts** collected from multiple vendors, each with unique formatting styles, varying image resolutions, and diverse print qualities. These receipts included both high-quality scans and camera-captured images, providing a realistic range of challenges for OCR processing.
- On average, **each receipt took approximately 2 minutes to process** when executed on a CPU-based system. This processing time included the entire pipeline — from image preprocessing (skew correction, noise removal, and orientation detection) to text detection, recognition, and parsing into structured fields.
- The performance evaluation of different extraction components revealed promising results:
- **Store Name Detection:** Achieved a detection accuracy of approximately **92%**, with most errors stemming from receipts that contained logos or stylized branding instead of plain text in uppercase format.

- **Date Extraction:** Reached a high accuracy of **96%**, benefiting from robust regular expression patterns that handled multiple date formats (dd/mm/yyyy and yyyy/mm/dd).
- **Total Amount Extraction:** Recorded an accuracy of **94%**, with minor misclassifications caused by receipts containing multiple “total” values (e.g., subtotal, total before tax, total after tax).

The main **sources of error** were:

- **Poor image quality** due to low resolution or motion blur during capture.
- **Faded text** on older thermal paper receipts, reducing OCR confidence.
- **Misaligned prints** where text was skewed beyond the correction threshold.
- **Highly unconventional store layouts**, where totals or dates were placed in unusual positions, making regex-based parsing less effective.
- For qualitative evaluation, a subset of **three sample receipts** was analyzed in detail, showing the pipeline’s ability to extract the correct store name, purchase date, total amount, and an itemized list in most cases. The visualizations and structured DataFrame confirmed the system’s usability for financial tracking and reporting.

6. Limitations

- Despite achieving a high degree of accuracy, the current implementation has several limitations that need to be addressed for broader adoption:
- **Processing Speed:**
The system currently relies solely on **CPU-based inference**, which significantly slows down the OCR process, especially when dealing with large batches of receipts. Without GPU acceleration, the 2-minute average per receipt processing time becomes a bottleneck for large-scale deployments.
- **Image Quality Sensitivity:**
OCR accuracy drops sharply when processing **damaged, smudged, or blurred receipts**. Such conditions are common with old receipts printed on thermal paper, which fades over time, making it harder for the recognition engine to distinguish characters.
- **Layout Variability:**
Retail receipts vary widely in design. Some receipts follow a standard structure, but others place important fields in non-traditional positions or use different formatting styles. This requires manual creation of **parsing rules** for new layouts, which reduces automation efficiency.
- **Single Language Support:**
The system is currently optimized for English-language receipts. Non-English text or bilingual receipts are either partially recognized or ignored entirely, limiting the system’s applicability in international contexts.

7. Future Scope

- To improve the robustness, scalability, and versatility of the system, several enhancements are proposed for future development:
- **GPU Acceleration for OCR:**
Migrating the inference process to GPU-based computation (using CUDA-enabled hardware) will significantly reduce OCR processing times, enabling near real-time recognition and analysis.
- **Address Standardization:**
Implementing an **address parsing and standardization module** with postal code recognition will improve the accuracy of location-based analytics. This would allow receipts from the same store chain to be grouped together even if their printed addresses vary slightly.
- **Multi-language OCR Support:**
Expanding the system to support **multi-language OCR models** will make it applicable for international users. This would involve integrating PaddleOCR multilingual models or combining with Google Vision OCR for broader character set coverage.
- **Enhanced Data Parsing with AI Models:**
Instead of relying solely on regex for field extraction, a machine learning-based **key-value pair extraction model** could be implemented to dynamically adapt to unseen layouts.
- **Web Dashboard Integration:**
A user-friendly **web-based dashboard** could be developed to provide real-time expense analytics, visualizations, and export options. Users could upload receipts, review parsed data, and download PDF summaries without running local scripts.

8. Code:

```
import re
import pandas as pd
from paddleocr import PaddleOCR
import matplotlib.pyplot as plt
import os

# Initialize OCR
ocr = PaddleOCR(use_angle_cls=False, lang='en')

# empty DataFrame
df_receipt_records = pd.DataFrame(columns=["File Name", "Store Name", "Address", "Date", "Items", "Total", "Raw Text"])

warnings.warn(warning_message)
Creating model: ('PP-LCNet_x1_0_doc_ori', None)
Using official model (PP-LCNet_x1_0_doc_ori), the model files will be automatically downloaded and saved in C:\Users\Work
space\paddlex\official_models.

Error displaying widget: model not found

Creating model: ('UVDoc', None)
The model(UVDoc) is not supported to run in MKLDNN mode! Using `paddle` instead!
Using official model (UVDoc), the model files will be automatically downloaded and saved in C:\Users\Workspace\paddlex\o
fficial_models.

Error displaying widget: model not found

Creating model: ('PP-OCRv5_server_det', None)
Using official model (PP-OCRv5_server_det), the model files will be automatically downloaded and saved in C:\Users\Worksp
ace\paddlex\official_models.

#SCANNING IMAGES USING OCR

def clean_total_amount(amount):
    if ',' in amount:
        amount.replace(',', '')
    if (amount[0] == '\n'):
```

```

def clean_total_amount(amount):
    if ',' in amount:
        amount.replace(',', '')
    if amount[0] == '\n':
        amount = amount[1:]
    return float(amount)

def process_receipt(img_path, i):
    print("scanning...")
    result = ocr.predict(img_path)
    lines = result[0]['rec_texts']

    full_text = "\n".join(lines)

    # Store Name (first line with only caps & symbols, no lowercase letters)
    store_name = next((l for l in lines if re.match(r'^[a-z]+$', l.strip())) , "Unknown")

    # Address (first 2-3 lines after store name that contain numbers + street keywords)
    address_lines = []
    address_keywords = ["JALAN", "ROAD", "STREET", "TAMPOI", "AVENUE", "BAHRU", "ALAM", "CITY", "TOWN"]
    for line in lines:
        if any(k in line.upper() for k in address_keywords):
            address_lines.append(line)
    address = ", ".join(address_lines) if address_lines else "Unknown"

    # Date
    date_match = re.search(r"\d{2}[/]\d{2}[/]\d{4}|\d{4}[/]\d{2}[/]\d{2}", full_text)
    date = date_match.group(0) if date_match else "Unknown"

    # Total Amount
    total_match = re.search(r"(\$|RM)?\s*\d+\.\d{2}", full_text)
    total_amount = total_match.group(0) if total_match else "Unknown"
    total_amount = clean_total_amount(total_amount)

    # Items (lines containing quantity & price)

    # Total Amount
    total_match = re.search(r"(\$|RM)?\s*\d+\.\d{2}", full_text)
    total_amount = total_match.group(0) if total_match else "Unknown"
    total_amount = clean_total_amount(total_amount)
    # Items (lines containing quantity & price)
    items = []
    item_pattern = re.compile(r".+\s+\d+(\.\d{2})?\s+(\$|RM)?\d+\.\d{2}")
    for line in lines:
        if item_pattern.match(line):
            items.append(line)
    items_text = "; ".join(items) if items else "Unknown"

    # Append to DataFrame
    global df_reciept_records
    df_reciept_records = pd.concat([df_reciept_records, pd.DataFrame([{"File Name": img_path,
        "Store Name": store_name,
        "Address": address,
        "Date": date,
        "Items": items_text,
        "Total": total_amount,
        "Raw Text": full_text
    }])], ignore_index=True)
    print(f"scanned image no.{i}\n")

# use this function to scan all the receipts in a folder
def scan_reciepts(folder):
    i = 0
    for file in os.listdir(folder):# file is the path of img
        print(f"remaining files to scan: {len(os.listdir(folder)) - i}")
        process_receipt(os.path.join(folder,file) , i)
        i = i +1

```

```
img_folder = "model_testing_images"
scan_receipts(img_folder)
#scanning takes average 1 min 40 secs per image
```

```
remaining files to scan: 12
scanning...
```

C:\Users\Workspace\AppData\Local\Temp\ipykernel_11136\170278602.py:44: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA entries is deprecated. In a future version, this will no longer exclude empty or all-NA columns when determining the result dtypes. To retain the old behavior, exclude the relevant entries before the concat operation.

```
df_receipt_records = pd.concat([df_receipt_records, pd.DataFrame([
scanned image no.0
```

```
remaining files to scan: 11
scanning...
scanned image no.1
```

```
remaining files to scan: 10
scanning...
scanned image no.2
```

```
remaining files to scan: 9
scanning...
scanned image no.3
```

```
remaining files to scan: 8
scanning...
scanned image no.4
```

```
remaining files to scan: 7
scanning...
scanned image no.5
```

```
remaining files to scan: 6
scanning...
scanned image no.6
```

```
] import matplotlib.pyplot as plt

def visualize_data(df):
    # Convert Total Amount to float for plotting
    df["Total"] = df["Total"].replace(r"^\d.", "", regex=True).astype(float)

    # Spending over time
    plt.figure(figsize=(8, 5))
    df.groupby("Date")["Total"].sum().plot(kind="bar")
    plt.title("Total Spending by Date")
    plt.xlabel("Date")
    plt.ylabel("Amount")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.savefig("spending_by_date.png")
    plt.close()

    # Spending by Store
    plt.figure(figsize=(8, 5))
    df.groupby("Store Name")["Total"].sum().plot(kind="bar")
    plt.title("Total Spending by Store")
    plt.xlabel("company")
    plt.ylabel("Amount")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.savefig("spending_by_store.png")
    plt.close()

visualize_data(df_receipt_records)
```

```

def _sanitize_for_paragraph(s, max_chars=200):
    import re
    if s is None: return ""
    s = str(s).replace('\r\n', '\n').replace('\r', '\n')
    s = re.sub(r'<br\s*/?>', '\n', s, flags=re.IGNORECASE)
    s = s.replace('&', '&').replace('<', '&lt;').replace('>', '&gt;')
    if max_chars and len(s) > max_chars:
        s = s[:max_chars] + "\n... [truncated]"
    s = s.replace('\n', '<br/>')
    return s

from reportlab.lib.pagesizes import A4
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Image, Table, TableStyle, PageBreak
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib import colors

def create_structured_pdf_only(df, filename="Receipt_Report.pdf",
                              drop_cols=["Raw Text", "FileName"],
                              truncate_cols=["Store Name", "Address"], max_chars=200,
                              chunk_size=30):

    import pandas as pd
    df2 = df.copy()
    df2 = df2.drop(columns=drop_cols, errors='ignore')
    for c in (truncate_cols or []):
        if c in df2.columns:
            df2[c] = df2[c].astype(str).apply(lambda s: s if len(s) <= max_chars else s[:max_chars] + " ...")
    if "Total" in df2.columns:
        df2["_Total_num"] = pd.to_numeric(df2["Total"].astype(str).replace(r"^\d\.", "", regex=True), errors='coerce')
    doc = SimpleDocTemplate(filename, pagesize=A4, leftMargin=20, rightMargin=20, topMargin=20, bottomMargin=20)
    styles = getSampleStyleSheet()
    small = ParagraphStyle('small', parent=styles['Normal'], fontSize=8, leading=10)
    title_style = styles['Title']

```

```

title_style = styles['Title']
elements = []
elements.append(Paragraph("Receipt OCR - Structured Report", title_style))
elements.append(Spacer(1, 8))
if "_Total_num" in df2.columns:
    total_spent = df2["_Total_num"].sum(skipna=True)
    elements.append(Paragraph(f"Total Receipts: {len(df2)}", small))
    elements.append(Paragraph(f"Total Spending: ${total_spent:.2f}", small))
    elements.append(Spacer(1, 8))
# Optional images (try/except)
try:
    elements.append(Paragraph("Spending Over Time", styles['Heading2']))
    elements.append(Image("spending_by_date.png", width=400, height=250))
    elements.append(Spacer(1, 8))
except Exception: pass
try:
    elements.append(Paragraph("Spending by Store", styles['Heading2']))
    elements.append(Image("spending_by_store.png", width=400, height=250))
    elements.append(Spacer(1, 8))
except Exception: pass
cols = list(df2.columns)
usable_width = A4[0] - doc.leftMargin - doc.rightMargin
for start in range(0, len(df2), chunk_size):
    chunk = df2.iloc[start:start+chunk_size]
    table_data = [[Paragraph(str(col), small) for col in cols]]
    for _, row in chunk.iterrows():
        row_cells = []
        for c in cols:
            val = row[c]
            safe_html = _sanitize_for_paragraph(val, max_chars=max_chars)
            row_cells.append(Paragraph(safe_html, small))
        table_data.append(row_cells)
    col_count = len(cols) if len(cols) > 0 else 1
    col_widths = [usable_width / col_count] * col_count

```

```

        row_cells.append(Paragraph(safe_html, small))
        table_data.append(row_cells)
    col_count = len(cols) if len(cols) > 0 else 1
    col_widths = [usable_width / col_count] * col_count
    tbl = Table(table_data, colWidths=col_widths, repeatRows=1)
    tbl.setStyle(TableStyle([
        ('GRID', (0,0), (-1,-1), 0.25, colors.grey),
        ('VALIGN', (0,0), (-1,-1), 'TOP'),
        ('FONTSIZE', (0,0), (-1,-1), 8),
    ]))
    elements.append(tbl)
    elements.append(PageBreak())
doc.build(elements)
print(f"Created structured PDF: {filename}")

```

```
create_structured_pdf_only(df_receipt_records)
```

9. Conclusion

This project successfully demonstrates an **end-to-end Receipt OCR pipeline** that scans, parses, and analyzes retail receipts into structured datasets. The generated reports provide valuable insights into spending behavior, enabling both individuals and businesses to manage expenses efficiently.

10. Git hub repository: <https://github.com/Ishan534/ReceiptScannerRepository>